

Justificativa da Escolha da Arquitetura em Camadas

Projeto Moneyball Scouting App — análise baseada na estrutura real do projeto

1. Nossa decisão

Como equipe, escolhemos a **Arquitetura em Camadas** como referência para organizar o nosso sistema Moneyball porque, ao analisar o projeto, percebemos que ele possui várias responsabilidades diferentes: autenticação e permissões, cadastro e gerenciamento de jogadores, equipes e partidas, registro de eventos de scouting, armazenamento de dados, cálculo de estatísticas e apresentação de rankings e dashboards.

Nosso sistema não é simplesmente um processo em que todos os dados entram por um ponto e precisam passar obrigatoriamente por uma sequência fixa até chegar a um resultado. O usuário pode acessar diferentes funcionalidades conforme sua necessidade. Por isso, entendemos que organizar o sistema por responsabilidades representa melhor o funcionamento da aplicação.

2. O que encontramos no nosso projeto

A estrutura do projeto confirma essa necessidade. Temos páginas específicas para jogadores, equipes, partidas, usuários, estatísticas e scouting, além de arquivos compartilhados para autenticação e funções do sistema. Também existe uma API específica para registrar eventos da partida e um banco de dados com entidades relacionadas entre si.

Parte do projeto	Função que identificamos
login.php / includes/auth.php	Autenticação, controle de acesso e perfis de usuário.
dashboard.php	Apresentação de indicadores, médias, ranking e últimas partidas.
jogadores/, equipes/ e partidas/	Cadastro, edição, exclusão, listagem, visualização e importação de dados.
scouting/registrar.php	Interface de scouting ao vivo, seleção de jogadores, eventos e coordenadas de arremessos.
api/registrar_evento.php	Recebimento, validação e persistência dos eventos da partida.
includes/functions.php	Funções compartilhadas, incluindo cálculos, rankings e operações utilizadas por diferentes páginas.
schema.sql	Estrutura persistente de usuários, equipes, jogadores, partidas, eventos, estatísticas e lineups.

3. Como isso se relaciona com as camadas

Para nós, a arquitetura em camadas permite pensar o projeto da seguinte maneira: a camada **de apresentação** reúne as telas e a interação com o usuário; a camada **de negócio** concentra as regras, validações e cálculos; a camada **de acesso a dados** fica responsável pelas consultas e alterações no banco; e o **banco de dados** mantém as informações persistidas.

No nosso projeto atual, essa separação ainda não está implementada de forma totalmente rígida. Por exemplo, algumas páginas PHP realizam consultas diretamente por meio de **\$pdo**, enquanto funções

compartilhadas ficam em `includes/functions.php`. Portanto, consideramos mais correto dizer que o projeto **possui uma organização compatível com a ideia de camadas, mas ainda pode ser refatorado para atingir uma arquitetura em camadas mais estrita**.

4. Um exemplo concreto: o registro de um evento

O melhor exemplo dessa decisão está no scouting. Em `scouting/registrar.php`, selecionamos um jogador e registramos ações como cesta de 2 pontos, cesta de 3 pontos, lance livre, assistência, rebote, roubo, toco, turnover e falta. A interface também permite marcar a localização do arremesso na quadra.

Depois, a página envia os dados para `api/registrar_evento.php`. Essa API valida se o usuário possui permissão, verifica se a partida e o jogador existem, verifica se o jogador pertence a uma das equipes da partida, grava o evento e recalcula as estatísticas. Quando o evento gera pontos, o placar também é atualizado.

Esse fluxo mostra claramente que existem responsabilidades diferentes. A interface coleta a ação do usuário; a API recebe e valida; as funções realizam operações de negócio; e o banco armazena os eventos e estatísticas. Isso é muito mais próximo de uma organização por camadas do que de um pipeline único que obrigatoriamente processaria todo o sistema em sequência.

5. Por que não escolhemos Pipeline como arquitetura principal

Também analisamos Pipeline, mas entendemos que ele representa melhor um **processo específico** do que a estrutura completa do nosso sistema. Em um pipeline, seria natural pensar em uma sequência como: entrada de dados → validação → transformação → processamento → saída.

No Moneyball, porém, existem diversos caminhos. Podemos fazer login e abrir o dashboard; cadastrar um jogador; editar uma equipe; importar partidas; consultar o ranking; comparar jogadores; ou registrar um evento durante uma partida. Essas operações não precisam passar todas pelas mesmas etapas e na mesma ordem.

Por isso, como equipe, concluímos que Pipeline seria mais adequado para representar algum fluxo interno de processamento, enquanto a arquitetura em camadas é mais adequada para representar a organização geral da aplicação.

6. Podemos ter Pipeline dentro do nosso sistema?

Sim. Essa é uma distinção importante. Não precisamos escolher entre as duas abordagens como se uma anulasse completamente a outra. Podemos manter a arquitetura geral em camadas e utilizar um pequeno pipeline em processos que realmente tenham etapas sequenciais.

Por exemplo, no registro de um evento podemos visualizar um fluxo semelhante a: **receber evento → validar dados → verificar partida e jogador → gravar evento → recalcular estatísticas → atualizar placar → retornar resposta**. Esse fluxo é sequencial, mas ele existe dentro de uma funcionalidade do sistema e não define sozinho a arquitetura completa da aplicação.

7. Vantagens para o nosso trabalho em equipe

A escolha também facilita o desenvolvimento em equipe. Como temos módulos diferentes, podemos dividir responsabilidades entre os integrantes sem que todos precisem alterar as mesmas partes do código. A separação também facilita testes e manutenção, porque uma alteração em uma regra de negócio não deveria exigir mudanças desnecessárias na interface.

Além disso, essa organização favorece o **Clean Code**, principalmente pela separação de responsabilidades. Quanto mais claramente cada parte do sistema tiver uma função definida, menor é a

chance de criarmos arquivos muito grandes e difíceis de manter.

8. Relação com o banco e as métricas do Moneyball

Nosso banco possui tabelas para **Usuario, Equipe, Campeonato, Jogador, Partida, Evento, Estatísticas e Lineup**. Essa estrutura mostra que o sistema trabalha com diferentes tipos de informação relacionados entre si. As estatísticas também possuem métricas como eficiência, eFG, TS, PPP, Offensive Rating, Defensive Rating e Regularidade.

Isso reforça nossa escolha porque a aplicação precisa buscar dados, aplicar regras e cálculos e apresentar resultados analíticos. Se misturássemos tudo em um único fluxo ou em páginas sem separação de responsabilidades, a manutenção ficaria mais difícil conforme o projeto crescesse.

9. Conclusão da equipe

Depois de analisar o projeto que desenvolvemos, concluímos que a **Arquitetura em Camadas** é a melhor escolha para representar a estrutura geral do nosso Moneyball Scouting App. Ela se encaixa nas diferentes áreas existentes no sistema e facilita a organização das funcionalidades, a manutenção, os testes, a divisão do trabalho entre a equipe e a evolução futura da aplicação.

Ao mesmo tempo, reconhecemos que a implementação atual ainda não apresenta uma separação de camadas totalmente rígida, pois algumas páginas PHP acessam o banco diretamente. Dessa forma, nossa justificativa não é dizer que o código atual já é uma arquitetura em camadas perfeita, mas sim que **essa é a arquitetura que melhor representa a direção estrutural que escolhemos para o sistema e que podemos aperfeiçoar por meio de uma refatoração**.

Em resumo: escolhemos Camadas porque nosso sistema possui várias responsabilidades e funcionalidades independentes. Pipeline pode ser utilizado dentro de processos específicos, mas não representa tão bem a aplicação completa. Para o nosso Moneyball, Camadas organiza melhor o sistema e cria uma base mais adequada para manutenção e crescimento.